



Department of Computing

Web Design and Development

Business Processes - Modelling & Design

Assignment: **Conceptualising the Problem Solution**
Due: **02nd March 2026**
Group: **7 Thursday**
Project Title: **Digital Examination Attendance System**

Instructions

- a) This document acts as a reference or template to aid you in the design and development of web products based on the proposed problem ideas.
- b) Fill in the respective sections with all necessary contents based on your group project ideas. Submit via Google Classroom as a soft copy and hand in a printed hard copy. A single submission is expected per group.
- c) Any form of plagiarism will attract severe penalties. **All rules of academic writing apply.**

d) Any other document **format** other than what is contained and specified herein will **NOT** be assessed. This cover page must not overflow to the next page.

Group Members

#	Registration Number	Name (<i>as indicated on the portal</i>)	Score	Score	Score
1	BSC/COM/23/24	Alfred Lonjezo			
2	BSC/COM/02/24	Tsokalida Nick			
3	BSC/52/24	Kaluluma Jessica Ndense			
4	BSC/COM/13/24	Nkhudzule Allan			
5	BSC/COM/05/24	Jamu Latisha			

Problem Definition

In many educational institutions, examination attendance is recorded manually using paper-based sheets during the exam session. Invigilators are required to move around the examination room to verify and mark each student's presence, introducing a range of operational and administrative challenges.

Project Goal/Objective

A digital attendance system is proposed in which student attendance is recorded before entering the examination room, shifting the verification process to the entry point.

Roles/Users

Teacher; Invigilators; Administrative / Academic Staff

Functional Requirements

- Authentication
 - o **FR1.1** An invigilator must be able to log in to the system.
 - o **FR1.2** Only authorised invigilators and administrators can access the system.
 - o **FR1.3** An administrator must be able to retrieve any user account by its unique identifier.
 - o **FR1.4** An administrator must be able to update user account details.

- o **FR1.5** An administrator must be able to delete a user account from the system.
- Session Creation
 - o **FR2.1** An administrator /teacher must be able to create an exam session.
 - o **FR2.2** An administrator / teacher must be able to upload or import a list of registered students for each session.
 - o **FR2.3** The system shall associate each student with a unique institutional ID.
 - o **FR2.4** Each session must be associated with a course.
 - o **FR2.5** The system must automate the process of changing session status depending on the schedule time planned by the user
- ID Scanning and Verification
 - o **FR3.1** An invigilator must be able to scan a student's institutional ID card at the exam entrance and at the end of the exam to mark completion.
 - o **FR3.2** The system must validate the scanned ID against the session's registered student list in real time.
 - o **FR3.3** The scan results (success or failure) must be displayed on screen within 5 seconds.
 - o **FR3.4** On successful verification, the system must mark the student as present and completed with a timestamp.

- o **FR3.5** If a certain student did not finish the exam for certain valid reasons, the system must allow invigilators to provide remarks for the individual.
 - o **FR3.6** The system must support bulk marking of multiple students as present in a single operation.
 - o **FR3.7** An invigilator must be able to update an existing individual attendance record.
 - o **FR3.8** An administrator must be able to retrieve all attendance records across all sessions.
- Instant Alerts
 - o **FR4.1** The system must display an on-screen notification for duplicated scan, invalid IDs and unregistered students.
- Offline Capability
 - o **FR5.1** In case of internet outage, the system must allow offline scanning and verification of students.
 - o **FR5.2** The invigilator must be able to manually search for student by name or ID number and mark attendance offline.
 - o **FR5.3** The system must store offline attendance records locally.
 - o **FR5.4** The system must automatically sync offline records to the central database when internet becomes available.
- Broken ID or Scanner Failure
 - o **FR6.1** If a student's ID card is broken, lost, or unreadable, an invigilator must be able to manually search for student by name or ID number.

- o **FR6.2** After manual verification, the invigilator must be able to mark the student as present.
 - o **FR6.3** If the scanner device fails, the invigilator must be able to use manual lookup as a fallback.
- Live Dashboard and reports
 - o **FR7.1** An administrator must be able to view a live dashboard showing total registered, present and absent.
 - o **FR7.2** An administrator must be able to view attendance for any active session at any time.
 - o **FR7.3** The system must generate per-session attendance reports including total present, total absent and timestamps.
 - o **FR7.4** The system shall produce attendance statistics (e.g., percentage present).
- Course Creation
 - o **FR8.1** An administrator or a teacher must be able to create , update and delete a course.
 - o **FR8.2** An administrator or teacher must be able to associate an existing course with an exam session during session creation
- Room Management
 - o **FR9.1** An administrator must be able to create a new examination room.
 - o **FR9.2** An administrator must be able to view a list of all registered rooms.
 - o **FR9.3** The system must allow retrieval of a specific room's details by its unique identifier.
 - o **FR9.4** An administrator must be able to update room details.
 - o **FR9.5** An administrator must be able to delete a room from the system.

- o **FR9.6** The system must support room lookup by its unique room code.
- o **FR9.7** The system must list all exam sessions associated with a specific room.

REST Endpoints

Module	Endpoint	Method	Description	Request Body / Parameters
Auth	/api/v1/auth/login	POST	Invigilator or admin login	{email, password}
	/api/v1/auth/register	POST	Register a new user account	{name, email, password, role}
	/api/v1/auth/profile	GET	Get the currently authenticated user's profile	-
	/api/v1/auth/{id}	GET	Retrieve a specific user account by ID	Path: {id}
	/api/v1/auth/{id}	PUT	Update a user account by ID	{name, email, role}
	/api/v1/auth/{id}	DELETE	Delete a user account by	Path: {id}

			ID	
Session	/api/v1/sessions	POST	Admin or teacher creates an exam session	{sessionName, venue, startTime, endTime, courseId}
	/api/v1/sessions	GET	List all sessions (supports ?status= filter: active, upcoming, expired)	-
	/api/v1/sessions/{sessionId}	GET	Retrieve details of a specific session	Path: {sessionId}
	/api/v1/sessions/{sessionId}	PATCH	Update details of a specific session	{title, venue, startTime, endTime}
	/api/v1/sessions/{sessionId}	DELETE	Admin or teacher (creator only) deletes a session	Path: {sessionId}

	/api/v1/sessions/{sessionId}/students	GET	List all students enrolled in a session	Path: {sessionId}
	/api/v1/sessions/{sessionId}/enrollments	POST	Upload or import registered student list for a session	CSV file or JSON array: {studentNumber, fullName}
Rooms	/api/v1/rooms	POST	Create a new examination room	{roomName, location, capacity}
	/api/v1/rooms	GET	Retrieve a list of all registered rooms	-
	/api/v1/rooms/{roomId}	GET	Retrieve details of a specific room by ID	Path: {roomId}
	/api/v1/rooms/{roomId}	PATCH	Update room details	{roomName, location, capacity}
	/api/v1/rooms/{roomId}	DELETE	Delete a room from the system	Path: {roomId}

	/api/v1/rooms/code/{roomCode}	GET	Look up a room by its unique room code	Path: {roomCode}
	/api/v1/rooms/{roomId}/sessions	GET	List all exam sessions associated with a room	Path: {roomId}
Attendance	/api/v1/attendance/mark	POST	Scan ID, validate student, and mark attendance (present/complete/deny entry)	{sessionId, institutionalId, scannerDeviceId}
	/api/v1/attendance/bulk-mark	POST	Bulk mark multiple students as present in a single operation	{sessionId, students: [{institutionalId}]}
	/api/v1/attendance/{id}	PUT	Update an existing individual attendance record	{sessionId, studentId, status, remarks}

	/api/v1/attendance	GET	Retrieve all attendance records across all sessions	Query: {sessionId, studentId}
	/api/v1/attendance/manual-search/{sessionId}	GET	Manually search for a student by name or ID when scanner fails	Path: {sessionId}, Query: {searchTerm}
Offline	/api/v1/offline/sync	POST	Push offline attendance records to server when connectivity is restored	{deviceId, offlineRecords: [{studentId, timestamp, status}]}
Dashboard	/api/v1/reports/{sessionId}	GET	Live dashboard: present/absent counts and full session attendance report	Path: {sessionId}
	/api/v1/reports/stats	GET	Retrieve overall attendance statistics across all sessions	-

Courses	/api/v1/courses	POST	Create a new course	{courseCode, courseName}
	/api/v1/courses	GET	List all courses	–
	/api/v1/courses/{courseId}	GET	Retrieve a specific course by ID	Path: {courseId}
	/api/v1/courses/{courseId}	PATCH	Update course details	{courseCode, courseName}
	/api/v1/courses/{courseId}	DELETE	Admin or teacher (creator only) deletes a course	Path: {courseId}